Equality from Monus and Neutral Major Premises

Two reductions for intuitionistic primitive recursive arithmetic

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

We obtain two different reductions. First, the tied-seed arithmetic of the bounded-initializer draft does not need a primitive Boolean test of actual equality. Two equations for truncated subtraction suffice. A gated complement identity proves equality reflection using only literal-equation induction, with its passive parameter fixed. A bounded selector then repairs the native logical bootstrap without assuming an unrestricted numerical selector. Together with the explicitly attributed bounded compiler, this gives a whole-domain bi-interpretation with full first-order intuitionistic primitive recursive arithmetic.

Second, a separate proof protocol for the existing initialized-iterator calculus uses a derived truth as the major premise of commands that previously had none. It removes the checker's optional-major branch. The induction mode is stored in the initializer field, whose falsity in the introduction branch preserves the restriction to literal equations. Complete implementations, precise input invariants, proof certificates, source measurements, and an audited comparison accompany this result.

The mathematical basis and the measured checker are different presentations. Their respective costs and novelty are not conflated. The measured family improves the inspected byte, token, and syntax-tree envelope, but different programs attain its coordinates. An absolute smallest axiomatization and a single implementation below every old coordinate have not been established. The written metatheorems have not been verified in Lean or Rocq.

Exact status. There is a new equality-reflection proof and a smaller complete implementation of the inherited native calculus. There is no basis for claiming the globally simplest theory or that the new monus basis itself has the smallest checker. The requested broad foundational and implementation breakthrough is therefore only partly achieved.

Contents

1 Results, comparison, and limits **3**

2 Every primitive of the two native presentations **5**

3 Replacing primitive numerical equality by two monus equations **7**

4 Native cases before a numerical equality characteristic **11**

5 Both interpretations and their composites **14**

6 The smaller checker and its complete boundary **15**

7 Why the absent major can be a proved empty array **18**

8 Why zero-source transport alone is insufficient **22**

9 Further research questions **23**

10 Reproduction and trust boundary **23**

A Attributed bounded initialization and finite function graphs **24**

B A compiler with internal bounds and cap coherence **25**

C Function graphs and preservation of induction **27**

D Whole-domain bi-interpretability **28**

E Internal order, difference, and addition under a bound **29**

F Complete measured source **34**

1 Results, comparison, and limits

The supplied repository was inspected at commit

`76a2651601d17b34d46753f189ea026a21ce8d29`.

Its `docs/drafts` directory contains 17 ZIP archives. The recursive inventory, including embedded historical archives, contains 32 distinct archive contents and 443 distinct Python source contents. Direct archive bytes were checked against their Git blob identities. The snapshot, recursive census, exact source identities, and current-checker replay results are included in `evidence/`.[1]

The incumbent complete-checker envelope is **558 normalized bytes, 262 significant tokens, and 296 abstract-syntax-tree nodes**. Different programs attain these values. All three leaders occur in the certified-zero package, including its ancillary profiles; older report headlines are not the current benchmark.[2]

Incumbent complete program	Bytes	Tokens	AST	Gzip-9
Certified zero, explicit list test	566	264	308	351
Certified zero, compact list test	558	264	310	345
Certified zero, generic traversal	598	262	296	350

The generic traversal has a documented qualification: nonempty malformed strings rely on the finite CPython recursion limit for rejection. It is not a structurally terminating parser on every possible JSON value in an ideal unlimited-stack execution. The list-based profiles avoid that qualification. All source and compression measurements include the complete Python module, its import, JSON input, and acceptance driver.

1.1 The mathematical reduction

The bounded-initializer draft gives a tied-seed recursor B and a primitive Boolean equality test ϵ. It explicitly asks whether the test's range and reflection axioms can be replaced by short recursive equations.[3] We give such a replacement, using monus d:

$$d(x,0) = x, \qquad d(x,Sy) = P(d(x,y)).$$

Here predecessor P is a fixed B-term, not a new primitive. A literal equation invariant proves that

$$\epsilon(x,y) := G\big(Z(d(x,y)), Z(d(y,x))\big)$$

is a Boolean characteristic of actual equality throughout the domain. The gate G and zero test Z are also B-terms. There is no appeal to an extensional program oracle, a standard-number calculation, or quantified induction in this bootstrap.

The corresponding native logical bootstrap is another necessary part of the result. The usual unrestricted numerical selector cannot simply be assumed in a tied-seed term language. We construct a selector whose first output is Boolean. It supplies contextual case splits for equation conclusions before equality reflection, and formula selection after reflection. This closes the logical dependency cycle.

The bounded compiler itself is inherited. Its cap coherence and finite quantifier-free graph method are reproduced with attribution in the appendices. Together with recurrence uniqueness for the new d, they give both whole-domain return interpretations. We do not label the compiler or its almost-unary term analysis as new.

1.2 The implementation reduction

Separately, the initialized-iterator native calculus can replace "optionally prove a major premise" by "always prove a major premise." When no major is needed, the premise is the empty-array representation of the already derivable truth $0 = 0$. The context initially contains that truth. A command still has four fields, but the case/progress mode moves to a field that is necessarily false-valued when introduction is selected.

This changes the proof protocol, not the established numerical I basis. The accompanying invariant rules out a false-valued numeric major that would otherwise collide with the checker's closure-match sentinel. The construction is therefore more than deleting an `if` statement: it requires a conservative truth assumption, mode relocation, a certificate translation, and a proof about every successful raw run.

Complete program	Bytes	Tokens	AST	Gzip-9
Incumbent separate minima	558	262	296	—
New explicit list test	550	263	307	349
New byte profile	542	263	309	343
New generic traversal	578	261	295	346
New separate minima	542	261	295	—

The three new envelope coordinates are attained by different programs. The explicit-list program is the clearest complete source; the byte profile inherits an exact-JSON list-test idiom; the generic profile has the same runtime qualification as its incumbent. No single program in this report is below all three incumbent minima. The new monus theory has not been assigned any of these checker sizes.

1.3 What "below the lower bound" can mean here

The draft sizes are achieved upper bounds on an unknown optimal implementation size. They are not proved lower bounds on all possible axiomatizations. We report exact improvements over their observed coordinate envelope under the unchanged normalizer. A theorem of absolute minimality would need a fixed description language and a separate lower-bound proof.

The two results are useful, but they do not justify announcing the globally simplest iPRA foundation or a large simultaneous implementation breakthrough. The remaining gap is concrete: implement the new arithmetic basis with all its logical and validation obligations, or find a single plain checker that improves every current coordinate.

1.4 Scope of iPRA and provenance

Here iPRA is the *full first-order intuitionistic* theory in a language indexed by fixed primitive-recursive descriptions, with their defining equations, successor boundary laws, and quantifier-free induction. Atomic induction is an equivalent familiar formulation in the full primitive-recursive language; Salehi uses that formulation.[8] We are not substituting the quantifier-free equational fragment for the full intuitionistic theory.

The corpus audit found no earlier neutral-major protocol with the mode relocation and live-context argument used here. It found an uncompleted suggestion to add monus to the tied-seed basis, and an explicit question about replacing equality reflection by recursion. Thus the new arithmetic claim is the completed gated-complement and native-bootstrap proof, not the first suggestion of the symbol

d. These novelty statements concern the inspected repository snapshot, not the entire mathematical literature.

All mathematical metatheorems below are written proofs. Executable checks cover finite examples, certificate rejection, and source accounting. They do not replace a Lean or Rocq proof of the metatheorems.

2 Every primitive of the two native presentations

The logical rules are shared. The numerical computation schemas differ, and that difference is kept explicit.

2.1 Numerical syntax

The new arithmetic basis has finite terms

$$t ::= x \mid 0 \mid St \mid d(t,t) \mid B(n;u,v.h).$$

Only u, v in h are bound. Its remaining variables are passive parameters, fixed during a recursion. Put $1 = S0$. The two recursion schemas are

$$B(0;u,v.h) = Sh(0,0), \tag{B0}$$
$$B(Sn;u,v.h) = h(n, B(n;u,v.h)). \tag{BS}$$

The monus schemas are

$$d(x,0) = x, \qquad d(x,Sy) = P(d(x,y)), \tag{M0, MS}$$

where

$$C(n,a) = B(n;u,v.a), \quad W(n) = B(n;u,v.u), \quad P(n) = W(C(n,n)).$$

All substitutions are simultaneous where appropriate and avoid capture.

The independently measured checker keeps the inherited language

$$t ::= x \mid 0 \mid St \mid I(n,a;v.h)$$

and its equations

$$I(0,a;v.h) = a, \qquad I(Sn,a;v.h) = h(I(n,a;v.h)). \tag{I0, IS}$$

The update receives its previous state, not the recursion counter. Its direct computation packets have the certified-zero syntactic restrictions specified in Section 6; equality transport restores all unrestricted instances. These I equations are not axioms of the new $B + d$ basis.

In both presentations, the binder notation abbreviates a staged ordinary first-order signature. Each fixed finite earlier-stage body introduces one function symbol whose arguments are its counter and passive parameters, together with the independent initializer in the I case. Bodies are never quantified over. Every term and proof uses finitely many stages. There is no internal uniformly total evaluator of arbitrary primitive-recursive codes.

2.2 Formula constructors and readings

Native formulas are generated by numerical equations and the paired binder:

$$A ::= (s = t) \mid D_{x,y}A.$$

The auxiliary closure Q_yA may be used in certificates and contexts, or admitted as a definable formula constructor. Its representative is $D_{a,y}A$ with unused a. Their ordinary readings are

$$Q_yA(y) := A(0) \to \forall y\, A(y), \qquad D_{x,y}A(x,y) := \exists x(A(x,0) \to \forall y\, A(x,y)).$$

These readings explain the interpretation; ordinary logical rules are not silently included among the native rules. The designated native contradiction is $0 = 1$.

Contexts Γ are finite reusable lists. All logical rules are:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma) \qquad \text{(Hyp)},$$

$$\frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)},$$

$$\frac{\Gamma \vdash Q_yA(y) \quad \Gamma \vdash A(0)}{\Gamma \vdash A(t)} \qquad \text{(QE)},$$

$$\frac{\Gamma \vdash D_{x,y}A(x,y) \quad \Gamma, Q_yA(b,y) \vdash F}{\Gamma \vdash F} \qquad \text{(DE)},$$

$$\frac{\Gamma, A(t,0) \vdash A(t,Sz)}{\Gamma \vdash D_{x,y}A(x,y)} \qquad \text{(DI-case)},$$

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)} \qquad \text{(DI-progress)}.$$

In Tr, F is any fixed native formula template. In DE, b is fresh for the retained context, conclusion, and fixed matrix parameters. In both introduction rules, z is fresh for the context, witness, and passive matrix parameters; the displayed assumption is discharged. **DI-progress is restricted to a literal numerical equation.** An equation beneath a binder does not satisfy that restriction.

The numerical computation schemas are axiom leaves. Equivalently, the implementation uses

$$\frac{\Gamma, H \vdash A}{\Gamma \vdash A} \qquad (H \text{ a permitted computation equation}). \qquad \text{(Supply)}$$

There are no additional native logical axioms, successor axioms, equality axioms, or induction schemas. In particular, reflexivity and explosion are derived below, not omitted costs hidden in a conventional logic.

Weakening, exchange, contraction, renaming, numerical substitution, and cut of an ordinary assumption are admissible. Their proofs are induction on finite derivations; eigenparameters are renamed before substitution or cut insertion. A numerical substitution leaves a literal equation literal. For syntactically restricted I computations, the certified-zero repair is applied whenever substitution creates a forbidden instance.

2.3 The ordinary comparison theory, with its entire logic

The next section first proves the arithmetic result in a conventional presentation T_{Bd}, to make the invariant easy to inspect. Its formula grammar is

$$A ::= (s = t) \mid \bot \mid A \wedge A \mid A \vee A \mid A \to A \mid \forall xA \mid \exists xA.$$

Its logical rules are exactly intuitionistic natural deduction:

Form	Rules
Assumptions	Use any member of the reusable context; weakening, exchange, and contraction.
$\bot$	From $\bot$ infer any formula; no introduction rule.
$A \wedge B$	Introduce from both conjuncts; eliminate to either conjunct.
$A \vee B$	Introduce from either disjunct; eliminate by two derivations of the same conclusion, under the respective disjuncts, discharging them.
$A \to B$	Introduce by discharging assumption A from a derivation of B; eliminate with a proof of A.
$\forall x A$	Introduce from $A(z)$ with fresh z absent from the retained context and fixed parameters; eliminate by capture-free term substitution.
$\exists x A$	Introduce from $A(t)$; eliminate under $A(z)$ with z fresh for the context, conclusion, and fixed parameters, discharging $A(z)$.
$=$	Reflexivity $t = t$; from $s = t$ and $F(s)$ infer $F(t)$ for every fixed formula template, with capture avoidance.

Its additional arithmetic principles are exactly the four displayed computation schemas, coverage, separation, and literal-equation induction listed next. The later native bootstrap derives the conventional logical interface rather than presupposing it.

Successor is eliminable from the native numerical signature: $S(a) = C(0, a)$. One can use this abbreviation everywhere, including the four computation schemas. Keeping S visible makes the proofs easier to read. This elimination is a definitional equivalence, not an additional measured checker reduction.

3 Replacing primitive numerical equality by two monus equations

We work initially in ordinary intuitionistic first-order logic with actual numerical equality. The exact numerical language consists of $0, S$, the staged tied-seed constructor B, and a binary symbol d. For each fixed finite earlier-stage body $h(u, v, \vec{p})$ the two B schemas are

$$B_h(0, \vec{p}) = Sh(0, 0, \vec{p}), \qquad B_h(Sn, \vec{p}) = h(n, B_h(n, \vec{p}), \vec{p}). \tag{B}$$

The two additional computation schemas are

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{M}$$

Here P is the fixed B-term constructed below; it is not a further primitive. Numerical coverage and separation are explicit:

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad 0 = S0 \to \bot. \tag{CS}$$

The sole induction schema is contextual induction on a literal equation:

$$\frac{\Gamma \vdash E(0, \vec{p}) \quad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p})}{\Gamma \vdash E(t, \vec{p})}, \tag{EI}$$

with z fresh for $\Gamma, t, \vec{p}$. All schemas are universally closed in their free numerical variables. There are no function variables or uniform evaluation symbols. Call this ordinary first-order theory T_{Bd}.

Coverage is an essential initial resource in the proof below. It must not be claimed as a consequence of the positive computation equations and literal-equation induction: the coordinatewise square of the standard model satisfies those positive equations and their equation-induction instances, while numerical coverage fails. A native calculus with a sound numerical case rule may supply this resource instead, but its reconstruction of ordinary logical rules must be justified independently.

3.1 Elementary terms available before equality reflection

Put $1 = S0$ and define

$$C(n,a) = B(n;u,v.a), \qquad W(n) = B(n;u,v.u), \qquad P(n) = W(C(n,n)), \qquad Z(n) = C(n,0).$$

The computation schemas alone give

$$C(0,a) = Sa, \quad C(Sn,a) = a, \quad W(0) = 1, \quad W(Sn) = n,$$

and therefore

$$P(0) = 0, \quad P(Sn) = n, \qquad Z(0) = 1, \quad Z(Sn) = 0. \tag{PZ}$$

Successor is injective, by application of P. It never equals zero: $Sx = 0$ would imply $x = P(0) = 0$, and hence $S0 = 0$, contradicting separation. Coverage and the endpoints of Z now prove

$$Z(x) = 0 \vee Z(x) = 1, \qquad Z(x) = 1 \iff x = 0. \tag{Z}$$

These are proofs in intuitionistic logic using the explicit coverage axiom; they do not use induction on a disjunction.

Define the one-sided gate

$$G(n,a) = P\big(P(B(a;u,v.C(Z(n),v)))\big).$$

Literal-equation induction on a proves

$$G(0,a) = 0, \qquad G(Sn,a) = a. \tag{G}$$

For the zero control the inner iterator has initial value 1 and identity update, so it is always 1. For a successor control its initial value is 2 and its update is successor, so its value at a is $S(Sa)$. Applying P twice gives the two equations. In particular, $G(1,a) = a$.

The inherited zero-start recursion macro is also available without numerical equality:

$$L_h(n,\vec{p}) = B\big(Sn;r,s.G(r,h(Pr,s,\vec{p}))\big).$$

Its first actual update is zero, and the subsequent updates are h; therefore its equations are

$$L_h(0,\vec{p}) = 0, \qquad L_h(Sn,\vec{p}) = h(n,L_h(n,\vec{p}),\vec{p}). \tag{L}$$

No arbitrary initialization has been assumed here.

3.2 Four preliminary monus identities

Lemma 3.1. *The theory proves, using only literal-equation induction,*

$$d(0,y) = 0, \tag{1}$$
$$P(d(Sx,y)) = d(x,y), \tag{2}$$
$$d(Sx,Sy) = d(x,y), \tag{3}$$
$$d(x,x) = 0, \qquad d(Sx,x) = 1. \tag{4}$$

Proof. For (1), induct on y using $P0 = 0$. For (2), keep x fixed and induct on y. The base is $P(Sx) = x$. At the step,

$$P(d(Sx,Sy)) = P(P(d(Sx,y))) = P(d(x,y)) = d(x,Sy),$$

where the middle equality applies P to the induction equation. Equation (3) follows immediately from (M) and (2). Finally induct on x in the two literal equations of (4). Their bases are $d(0,0) = 0$ and $d(1,0) = 1$, and their steps are exactly (3). No induction hypothesis is generalized or specialized to a changed passive parameter. □

3.3 The gated complement identity

The key step is an equation invariant that permits the ordinary conditional complement argument before quantifier-free induction is available.

Lemma 3.2 (Gated complement)**.** *For all x, y the theory proves*

$$G(Z(d(y,x)), d(x, d(x,y))) = G(Z(d(y,x)), y). \tag{GC}$$

Consequently,

$$d(y,x) = 0 \quad \longrightarrow \quad d(x, d(x,y)) = y. \tag{C}$$

Proof. Keep x fixed and apply literal-equation induction to (GC) as an equation in y. At $y = 0$, the gate control is $Z(d(0,x)) = 1$, and both output arguments are zero by $d(x,0) = x$ and $d(x,x) = 0$.

For the step, retain the induction equation and set $a = d(Sy, x)$. Use coverage on a. If a is a successor, $Z(a) = 0$ and both sides of the new equation are zero by (G). It remains to consider $a = 0$.

By (2),

$$d(y,x) = P(d(Sy,x)) = 0.$$

Thus the induction equation, with both gates simplified at control 1, gives

$$d(x,t) = y, \qquad t = d(x,y). \tag{IH}$$

If $t = 0$, then $d(x,t) = x$, so $x = y$. But this makes $a = d(Sy, y) = 1$, contradicting $a = 0$. Hence $t \neq 0$. Coverage therefore provides u with $t = Su$. The monus successor schema identifies

$$u = P(t) = d(x, Sy).$$

Put $v = d(x,u)$. Equation (IH) and the successor schema give

$$P(v) = P(d(x,u)) = d(x, Su) = d(x,t) = y.$$

If $v = 0$, then $y = 0$. Hence $t = d(x,0) = x = Su$, and

$$v = d(x,u) = d(Su,u) = 1,$$

a contradiction. Thus $v \neq 0$, and coverage gives $v = Sz$. Since $P(v) = y$, the predecessor equations give $z = y$, so $v = Sy$. Together with $u = d(x, Sy)$ this is

$$d(x, d(x, Sy)) = Sy,$$

which is exactly the required step with gate control $Z(a) = 1$. All contradictions use only $0 \neq 1$ and intuitionistic falsity elimination. All case splits occur inside the step derivation with the original induction equation retained. The only induction matrix used was the single literal equation (GC).

Finally if $d(y,x) = 0$, the control of both gates in (GC) is 1, giving (C). □

Corollary 3.3 (Antisymmetry before order theory)**.** *The theory proves*

$$d(x,y) = 0 \ \wedge \ d(y,x) = 0 \quad \longrightarrow \quad x = y.$$

Proof. Equation (C) gives $d(x, d(x,y)) = y$. Substituting $d(x,y) = 0$ and then $d(x,0) = x$ gives $x = y$. □

3.4 A characteristic of actual equality

Define the fixed numerical term

$$\epsilon(x,y) = G\big(Z(d(x,y)), Z(d(y,x))\big). \tag{E}$$

Theorem 3.4 (Whole-domain equality reflection)**.** *The theory T_{Bd} proves*

$$\epsilon(x,y) = 0 \vee \epsilon(x,y) = 1, \qquad \epsilon(x,y) = 1 \iff x = y.$$

These statements concern actual equality throughout the domain.

Proof. Each input of the gate is zero or one by (Z). The two gate equations prove the displayed range. They also prove

$$\epsilon(x,y) = 1 \quad \iff \quad d(x,y) = 0 \wedge d(y,x) = 0 :$$

if the first gate input were zero, the output would be zero; when it is one, the output is the second input. Use $0 \neq 1$ and (Z) in both directions. Antisymmetry proves the right-to-left numerical identification required for reflection. Conversely $x = y$ implies both differences are zero by $d(x,x) = 0$ and equality substitution, so the gate output is one. □

In particular numerical equality is decidable, but excluded middle for arbitrary quantified formulas has not been postulated. Every fixed quantifier-free formula has a Boolean characteristic term: use (E) for atoms, zero for falsity, Z for negation, and G for conjunction; disjunction and implication follow from the Boolean identities. Implication is legitimate intuitionistically because the antecedent is decidable. Literal-equation induction on the characteristic being one yields contextual quantifier-free induction.

3.5 Consequences for the existing bounded compiler

The equality term just proved satisfies precisely the range and actual reflection axioms of the inspected $B + \epsilon$ bounded-initializer presentation. Its finite descriptions and literal-equation induction instances translate by substitution of (E). Therefore the existing bounded-initialization construction, cap-coherence argument, and finite quantifier-free function graphs apply to T_{Bd}. They give a whole-domain interpretation of iPRA with actual equality.

In the reverse direction, iPRA interprets each fixed B_h by primitive recursion with base $Sh(0,0,\vec{p})$ and step h, and interprets d by ordinary truncated subtraction. It proves the two monus equations, coverage, separation, and every source equation-induction instance. The common numerical domain is unchanged.

For the source-side composite, first identify the d-free constant-body C and counter-body W recursors by recurrence uniqueness, and hence their predecessor macro P. The imported primitive-recursive monus graph is provably equal to the original d: induct on its second argument using its defining equations and (M). When a graph-defined function is used in this comparison, expand its finite quantifier-free graph cases; the quantifier-free induction already derived above licenses this definitional-extension argument. For each fixed staged B symbol, recurrence uniqueness similarly identifies the composite with the original symbol, proceeding by external induction on the finite description. The iPRA-side composite follows from primitive- recursion uniqueness in the same way. Thus the inherited compiler proof extends to full-domain bi-interpretability and equiconsistency.

This is a new arithmetic presentation with a transparent reflection bootstrap. It is not, by itself, a proof-checker size reduction. Its ordinary logical rules, coverage, separation, all four computation

schemas, and equation induction must be included in any honest implementation comparison. A checker for a different inherited calculus cannot be assigned to this new theory merely because both are bi-interpretable with iPRA.

4 Native cases before a numerical equality characteristic

There is a potential circularity in importing the preceding arithmetic proof directly into the native Eq/Q/D calculus. Its theorem-level numerical case rule is not automatically a contextual split on $t = 0$ and $t = Sz$ when the retained induction hypothesis mentions t. Moreover, the tied-seed arithmetic has no unrestricted numerical selector among its term functions. The following bounded selector removes that circularity. Only hypothesis use, full formula transport, the native Q/D rules, the B computation schemas, and literal-equation progress are used. No ordinary implication, conjunction, disjunction, or quantifier rule is assumed in this section.

4.1 Initial native resources

The native rules derive numerical cases and literal-equation induction with their usual freshness conditions, using an unused witness in D, then D elimination and Q specialization. The computation $C(S0, p) = p$, used as both premises of transport with motive $w = p$, gives $p = p$. Symmetry, transitivity, and numerical congruence follow from the usual transport motives. The B macros P, Z, G of the previous section are therefore available by their same equation derivations.

Equation explosion is also available. Under $0 = 1$, transport between $C(0, a) = Sa$ and $C(1, a) = a$ gives $Sa = a$ for arbitrary a. Literal-equation induction on $a = 0$ then proves every $a = 0$ in this retained context. Hence every numerical equation follows. External induction on a fixed native formula, using case introduction for each D and the unused-witness representation of Q, gives explosion for all native formulas. This is an external finite proof transformation; it is not compound-formula induction in the object theory.

For a numerical term b, say $\mathrm{FR}_{\{0,1\}}(b)$ holds when, for every fixed native formula template K, the following metarule is available in an arbitrary retained context:

$$\Gamma \vdash K(0), \quad \Gamma \vdash K(1) \quad \Longrightarrow \quad \Gamma \vdash K(b). \tag{FR}$$

The context may mention b; the premises add no equations $b = 0$ or $b = 1$. Native numerical cases on the fresh template $w \mapsto K(Zw)$, using $Z0 = 1$ and $Z(Sw) = 0$, prove $\mathrm{FR}_{\{0,1\}}(Zt)$ for every t. In particular, $Z(Zb) = b$ follows for every such finite-range b by applying FR to that equation's two numeral instances.

4.2 A selector whose first output is Boolean

Define

$$J(a, b) = C(Zb, a), \qquad H(y, b, u) = J\big(G(y, u), G(Zy, b)\big). \tag{H}$$

Direct computations give

$$J(a, 0) = a, \quad J(a, 1) = Sa, \qquad H(0, b, u) = b \quad (b \in \{0, 1\}), \quad H(Sy, b, u) = u. \tag{H endpoints}$$

The first endpoint also holds for any term b with the displayed functional range rule: apply that rule to the fixed equation template. The successor endpoint holds without any restriction on b.

This is not an unrestricted numerical selector. Its zero output is Boolean; its successor output is arbitrary. Consequently its existence does not contradict the one-coordinate majorant bound for tied-seed terms. It is exactly enough for contextual Boolean case elimination.

Lemma 4.1 (Contextual Boolean cases for equation conclusions)**.** *Suppose* $\mathrm{FR}_{\{0,1\}}(b)$ *holds. If*

$$\Gamma, b = 0 \vdash U = V, \qquad \Gamma, b = 1 \vdash U = V,$$

then $\Gamma \vdash U = V$ *is derivable in the native calculus. The retained context and the conclusion may mention* b*.*

Proof. With fresh a, y, define

$$K(x) = D_{a,y}(H(y, x, U) = H(y, b, V)),$$

where a is unused. For $r = 0, 1$, use case introduction to prove $K(r)$. Its local zero assumption reduces to $r = b$, by the Boolean endpoint of H for r and for b. Symmetry gives $b = r$, so cut in the corresponding supplied branch proof. The successor matrix is $U = V$ by the other endpoint of H. This constructs $\Gamma \vdash K(r)$ for both numeral values, discharging the local equation.

Apply FR to obtain $\Gamma \vdash K(b)$. Paired elimination opens its conditional closure. Its zero instance is $b = b$, supplied by reflexivity and the endpoint computations. Specialization at 1 returns $U = V$. All temporary variables are fresh, and all temporary assumptions have been discharged. □

4.3 Contextual zero/successor splits for arbitrary terms

Native numerical cases, applied to fresh variables in the following literal-equation templates, prove

$$G(Zt, t) = 0, \qquad G(Z(Zt), t) = G(Z(Zt), S(Pt)). \tag{Reconstruction}$$

For the first identity the zero branch is $G(1, 0) = 0$ and every successor branch has control zero. For the second, the zero branch has control zero, while the successor branch has control one and $P(Sx) = x$. There is no substitution into a retained context.

Lemma 4.2 (Contextual numerical cases for equations)**.** *Let* z *be fresh for* Γ, t, U, V*. If*

$$\Gamma, t = 0 \vdash U = V, \qquad \Gamma, t = Sz \vdash U = V,$$

then $\Gamma \vdash U = V$ *is derivable natively.*

Proof. Apply Lemma 4.1 to $b = Zt$. Under $Zt = 1$, the first reconstruction identity yields $t = 0$, so the first given branch applies. Under $Zt = 0$, the second identity has control $Z0 = 1$, so it yields $t = S(Pt)$. Substitute Pt for the fresh z in the second given branch and cut in this equation. The conclusion is the same equation $U = V$ in both Boolean branches. □

Every case split in the gated-complement proof has an equation as its ultimate conclusion. The zero cases that contradict $0 \neq 1$ are replaced in the native calculus by the equation-explosion transformation above. Thus the proof of the gated-complement invariant, antisymmetry, and the two directions of equality reflection translates natively without importing ordinary logical rules. Its only induction matrix remains the single equation in the displayed invariant.

4.4 Reconstructing ordinary logic after reflection

The equality term $\epsilon(s,t)$ now has the native functional Boolean range rule and satisfies

$$s = t \;\dashv\vdash\; \epsilon(s,t) = 1$$

in every retained context. Its finite range follows from the two functional range rules for its Z inputs and the gate endpoints. For an atomic formula $E \equiv (s = t)$ define a gate by

$$\mathcal{G}_v(E) \;:\equiv\; G(v, \epsilon(s,t)) = G(v,1).$$

It is provable at $v = 0$ and interderivable with E at each $v = Sw$. For atomic $E \equiv (s = t)$ and $F \equiv (r = w)$ define a formula selector

$$\mathcal{M}_v(E,F) \;:\equiv\; H(v, \epsilon(s,t), \epsilon(r,w)) = 1.$$

The Boolean endpoint of H gives $\mathcal{M}_0(E,F) \dashv\vdash E$; its successor endpoint gives $\mathcal{M}_{Sz}(E,F) \dashv\vdash F$. The zero branch is bounded, so no unrestricted numerical selector has been introduced.

Extend the gate through D, replace Q by its unused-witness D representation, and align binder depths using the usual gate padding. Then extend the formula selector through matched D binders by external recursion on their finite syntax. Binder congruence uses only the native case rule, paired elimination, transport, and ordinary-assumption cut, so it is already available. These are the same structural logical constructions as in the inherited Choice-DI bridge [2], with the atomic selector just given replacing its unbounded numerical selector. Both inherited derivations are included under `reference/initialized_original/` as `inherited_choice_basis.tex` and `inherited_ordinary_bridge.tex`.

For explicitness, put $p(z) = L(z; u, v.Zv)$, so $p(1) = 1$ and $p(2) = 0$. With all unused coordinates fresh, the ordinary macros are

$$\begin{aligned}
A \Rightarrow_* B &= D_{a,z}\mathcal{M}_z(A,B),\\
A \wedge_* B &= D_{a,z}\mathcal{G}_z\mathcal{M}_{p(z)}(A,B),\\
\exists_* x\, A(x) &= D_{x,y}\mathcal{G}_y A(x),\\
A \vee_* B &= \exists_* x\, \mathcal{M}_x(A,B),\\
\forall_* x\, A(x) &= A(0) \wedge_* D_{a,z}\mathcal{G}_z A(z),\\
\bot_* &= (0 = 1).
\end{aligned}$$

Implication introduction uses the matrix's zero assumption; elimination opens its closure and specializes at one. The gate's provable zero instance supplies conjunction and existential introduction, while specialization at one or two supplies their eliminations. After implication is available, ordinary contextual numerical cases for arbitrary formulas follow by applying Native NC to an implication template; these cases justify disjunction elimination and universal elimination. All freshness and capture-avoidance conditions are the inherited ones.

These standard native macros for implication, conjunction, disjunction, existence, universal quantification, and falsity therefore have their usual intuitionistic introduction and elimination rules. Both formula composites follow from the same gate and selector endpoint proofs. After this stage the ordinary T_{Bd} argument and the inherited bounded compiler may be used without a logical bootstrap cycle.

5 Both interpretations and their composites

The preceding argument supplies an actual numerical equality characteristic and full intuitionistic logic. The rest of the arithmetic translation uses the bounded-initializer construction, reproduced in Appendix A. This section explains exactly what is inherited and what must additionally be checked for the new primitive monus.

5.1 Why graphs, rather than unrestricted output terms

Tied-seed terms can still be bounded by a primitive-recursive unary function of one input coordinate. That does not prevent a function from being definable by a finite choice of terms. The bounded compiler supplies, for every fixed primitive-recursive description f, source terms

$$\tau_f(b, \vec{x}), \qquad M_f(b),$$

with proved bounds and coherence: whenever both b and c bound the arguments, $\tau_f(b, \vec{x}) = \tau_f(c, \vec{x})$. Neither f nor a description code is quantified over inside the source theory.

For a nonempty tuple $(x_1, \ldots, x_k)$, let $C_i(\vec{x})$ say that i is its first largest coordinate in the internally derived discrete order. These are finitely many exhaustive, disjoint, quantifier-free cases. The graph

$$\Phi_f(\vec{x}, z) = \bigvee_{i=1}^{k} \left(C_i(\vec{x}) \wedge z = \tau_f(x_i, \vec{x})\right)$$

is provably total and single-valued. Its outputs are explicit terms on each case. Nullary functions have a single fixed value term.

Finite value cases are crucial for induction preservation. Expanding every occurrence of a graph-defined function in a finite term gives a finite family of quantifier-free guards and source value terms. Refining these cases for every term in a quantifier-free formula gives an equivalent source quantifier-free formula. This applies to implication and negation as well, because the guards cover all inputs and the values agree on overlaps. Source quantifier-free induction, already derived from literal-equation induction and our equality characteristic, therefore proves every translated iPRA induction instance.

An arbitrary existential graph would not establish this fact. The finite term-value construction in the appendix is the relevant part of the inherited proof.

5.2 Arithmetic round trips

Interpret the new $B + d$ language in iPRA by ordinary primitive recursion: the initial value for B_h is $Sh(0, 0, \vec{p})$, its update is h, and d is ordinary truncated subtraction. The two B schemas and the two monus schemas hold. Interpret native equations as actual equality and Q, D by their ordinary readings. Coverage proves DI-case and equation induction proves DI-progress. Every native rule is therefore sound.

Interpret iPRA back by the finite graphs above. The computation and finite-case lemmas prove the defining equations and all induction instances, and predecessor plus separation prove the successor boundary laws. Both numerical interpretations have domain formula $x = x$ and use actual equality.

On the iPRA side, external induction on each fixed primitive-recursive description, with internal quantifier-free induction at a recursion node, identifies its composite with the original function.

On the source side, first preserve the d-free constant-body recursor C and counter-body recursor W by their equations and recurrence uniqueness. This preserves $P = W(C(n, n))$. Next preserve

primitive d using its two equations, the preserved P, and finite quantifier-free graph elimination. The constructed difference $\widehat{d}$ of the inherited bridge has the same two recursive equations as our primitive d. Literal-equation induction in its second argument proves $\widehat{d}(x, y) = d(x, y)$; this source-level proof is included at the end of the order appendix. Thus the new primitive is recovered, not just some externally equivalent standard function. The equality characteristic is then identified by its proved Boolean range and reflection. For each fixed staged B_h, external induction on its earlier-stage body identifies the two updates, and equation induction on the counter identifies their results. When a graph-defined function occurs in a comparison, its finite quantifier-free cases justify the induction in the original language.

5.3 Formula round trips

The gate and selector endpoint proofs identify each ordinary connective with its native macro. The nontrivial native composite is

$$D_{x,y}A(x, y) \quad \dashv\vdash \quad \exists_* x\big(A(x, 0) \Rightarrow_* \forall_* y\, A(x, y)\big).$$

Forward, open the paired formula at a fresh witness. Under its zero instance, QE provides every instance, giving the universal and then the implication and existential macros. Reverse, eliminate the translated existential, retain the matrix's zero instance, and use its implication and universal at Sz. Case introduction reconstructs the original paired formula. The unused-witness representative gives the analogous composite for Q.

External structural induction on a fixed formula now proves both formula composites, including arbitrary quantified formulas. All quantifiers range over the original whole domain. The identity relation $x = y$ defines each numerical isomorphism and preserves every operation, relation, and translated formula. Ordinary falsity corresponds to the native contradiction $0 = 1$, whose explosion was explicitly derived.

Theorem 5.1 (The new native arithmetic basis)**.** *The native Eq/Q/D theory with B0, BS, M0, and MS, and exactly the rules of Section 2, is whole-domain bi-interpretable and equiconsistent with full first-order iPRA. Both composite domain isomorphisms are defined by actual equality.*

The equiconsistency conclusion follows from effective proof translations preserving contradiction. It is not a claim that either theory proves its own consistency. Nor does it identify the implementation cost of this basis with the smaller, independently measured I checker.

6 The smaller checker and its complete boundary

This section concerns the initialized-iterator presentation specified in Section 2. It does not implement the new $B + d$ grammar. The mathematical theorem is identity preservation of its native consequence relation under a changed proof protocol. Thus its existing whole-domain iPRA interpretations transfer unchanged.

6.1 A precise sorted representation

The inherited encoding is sorted: the same array can have different meanings in a numerical position and a formula position. Its native meanings are:

Array arity	Numerical meaning	Formula meaning
0	0	$0 = 0$
1	Successor of its child	Paired binder D
2	0, annotations ignored	Equality of its two terms
3	Initialized scalar iteration	Closure Q, last child is matrix
≥ 4	0, annotations ignored	$0 = 0$, annotations ignored

Canonical paired formulas are `[B]`, canonical equations are `[s,t]`, and canonical closures are `[B,B,B]`. The three copies let list multiplication export exactly a closure when its major is unary; all other nonempty array majors export a high-arity truth. This arity construction is inherited, not a new contribution.[5]

The opener increases depth by $5 - r$ at an array of arity r. Numerical root depth is three; formula root depth is zero. Under p protected numerical slots, the corresponding depths are $3 + 2p$ and $2p$. Root coordinates $0, 1, 2$ denote numerical zero. Coordinates $3 + 2j, 4 + 2j$ denote the jth protected slot. Negative admitted labels name rigid parameters. A unary successor's child is decoded beneath two dummy zero slots; the counter and initializer of an iterator are decoded beneath one dummy zero slot, while its update is under its actual state slot. This is how the common depth law implements the different sorted constructors.

At global floor k, a scalar in $[k, d)$ is retained. A scalar escaping at numerical depth is the opening hole; its supplied image is checked at numerical root depth three. An escaping scalar at formula depth erases to the truth padding. A missing image at a genuinely used numerical hole rejects. The proof that this implements capture-free opening is the inherited structural induction over the sorted decoder. An image has only rigid negative free labels, so copying it under protected slots does not capture those free parameters. Large ignored arities retain their constant meanings; they are not assumed to be idempotently revalidated as raw trees.

Every non-hypothesis continuation decreases k. Incoming goals and premises have been checked at the older floor, so a newly admitted parameter is fresh for the retained context and conclusion. If a finite raw certificate admits fractional labels in a newly opened interval, they are distinct fresh parameters; all are included in a later zero-instance substitution when needed.

6.2 The restricted direct computation interface

The initializer field must be nonempty to select computation. A false predecessor selects the zero equation; a true predecessor selects its successor equation. Thus the canonical directly supplied instances are

$$I(0, a; v.h) = a \quad (a \text{ not literal } 0),$$

$$I(Sn, a; v.h) = h(I(n, a; v.h)) \quad (n, a \text{ not literal } 0).$$

This is a syntactic restriction, not a numerical nonzero test. The inherited normalization is short and noncircular. Set

$$Z_* := I(S(S0), S0; v.0).$$

The permitted successor equation immediately gives $Z_* = 0$: its predecessor and seed are both the nonempty numeral one. For an excluded computation, replace every offending predecessor or seed position simultaneously by a fresh placeholder w in a fixed equation motive $E(w)$. Keep unrelated zeros in the update unchanged. Then $E(Z_*)$ is a permitted computation, and transport along $Z_* = 0$ proves $E(0)$. The repair consists of one transport node and its two computation premises. All unrestricted I0 and IS instances follow.[2]

Generated raw hypotheses need only have a known zero instance. For fresh $\vec{b}$ absent from Γ, A, the rule

$$\vdash H(\vec{0}), \quad \Gamma, H(\vec{b}) \vdash A \quad \Longrightarrow \quad \Gamma \vdash A$$

is admissible: substitute zero for the fresh names in the second derivation, rename subsequent eigenparameters first, and cut in the first proof. The modified computation mode still uses this rule. It would be false to say every generated open hypothesis is uniformly true in its newly admitted parameters.

6.3 All fields and all command branches

The input is one JSON array. Its last element is the requested goal. Earlier four-field packets $[m, s, u, t]$ are consumed from the end. The initial assumption is the empty array, a representation of the derived theorem $0 = 0$. A hypothesis closes without consuming a packet. Every other command first proves m in the current context.

Case	Meaning
Truthy m	Check the substantive major. The old equality/closure endpoint match can change the goal from $F(u)$ to $F(s)$. Export $m * 3$ as the paired elimination resource or a constant truth alias.
False m, true u	Supply a computation. Field s is the predecessor, u the initializer, and t the update.
False m, false u	Introduce the unary paired goal with witness t. Canonical $u = 0$ selects progress for a literal equation; canonical $u = []$ selects its zero-case premise. Field s is unused.

Every canonical absent major is `[]`; it closes immediately against the initial truth assumption. Scalar zero is deliberately not an alternative absent major. The next section proves why this detail makes the unguarded matcher safe.

6.4 Cost and verification

Relative to their corresponding frozen predecessors, the explicit and byte profiles each remove 16 normalized bytes, one token, and one AST node. The generic profile removes 20 bytes, one token, and one AST node. The changes to the mathematical protocol, rather than unrelated import or driver edits, account for these matched differences.

Against the independent old envelope, the new byte profile goes from 558 to 542 bytes, a reduction of approximately 2.87%. The explicit list-based program is 550 bytes. The generic traversal reaches 261 tokens and 295 AST nodes, improving each old structural minimum by one; its 578 bytes do not beat 558. Consequently no single simultaneous three-coordinate record is asserted.

The normalizer removes actual docstrings, applies `ast.unparse`, and appends a newline. It counts the entire module. The delivered run uses CPython 3.12.14 and zlib 1.3.2. Gzip measures `gzip.compress(source, compresslevel=9, mtime=0)`, including its wrapper. Raw and normalized measurements, source hashes, and the actual gzip streams are supplied. Compression is an informational measurement.

The offline checker suite runs 46 positive, negative, alias, and malformed cases through the three compact programs and the readable expansion, under both ordinary Python and `-O`: 368 expected process results. Two additional mutation controls replace the correct mode comparison with the tempting wrong one, and demonstrate acceptance of a forbidden compound-progress certificate. The corrected checker rejects that certificate. This checks the actual restriction whose loss would matter, rather than only a generic malformed input.

The finite arithmetic diagnostics separately evaluate the displayed B macros and two monus equations on inputs through eight, including the intermediate equalities of every active gated-complement step. They perform 1,750 assertions and distinguish seven failure mutants. Those diagnostics are not a full proof checker for $B + d$ and are not included in the compact trusted-source measurements.

7 Why the absent major can be a proved empty array

This section audits the new control-flow change on raw input, including noncanonical JSON. It uses the inherited sorted arity decoder and semantic-opening lemmas for the numerical and logical rules. It does not replace those lemmas with finite testing or claim proof-assistant certification.

7.1 The exact change

The initial context is the one-element list `[[]]`. Its sole formula, the empty array, denotes $0 = 0$ under the inherited decoder. At every non-hypothesis command the checker now executes

```
m, s, u, t = q()
c(m, g)
```

without testing whether `m` is nonempty. A canonical command with no substantive major puts `[]` in this field. Its premise closes immediately by hypothesis, without consuming a command or changing the fresh-name floor.

The final choice of the retained assumption uses

```
(m or len(a) % 2 == u) and k
```

instead of comparing the arity parity with `m`. In an introduction command, `u=0` selects equation progress and `u=[]` selects the zero-case premise. This reuse is legitimate because introduction is reached only when both `m` and `u` are false-valued. Computation still requires a truthy initializer `u`; its missing literal-zero cases have the inherited certified-zero repair.

The source is deliberately left with the old matcher

```
m in ([s, u], not s and [t] * 3)
```

and no added `m and` guard. As an isolated Python expression this would be unsafe: if `s` is truthy, its second alternative is `False`, which compares equal to numeric zero. The following whole-program invariant proves that this dangerous major cannot reach the matcher.

7.2 Two facts about the root opener

Call a Python value *numeric* here when it is an integer, a float, or a Boolean returned by the JSON loader. The finite Python execution also admits the loader's optional nonfinite float values. No custom numeric class is supplied by a certificate.

Write $O_k(r; t)$ for the opener on the numeric value r, with image t and its default formula-root depth zero. Direct evaluation of the program gives

$$O_k(r; t) = \begin{cases} r, & k \le r < 0, \\ 0, & \text{otherwise.} \end{cases} \qquad \text{(Root scalar)}$$

In particular this result is numeric and independent of the image t. The formula covers negative allocated names and negative fractional names without identifying different numerical values. Signed zero and Booleans have Python's ordinary numerical equality behavior. A NaN fails the interval test and returns zero; the relevant infinite values also return zero. None of these results is a list.

A second fact is that a successful root opening of a list returns a list. Under either strict list predicate, no other input type can return a numeric result except a numeric input. Under the generic iterable opener, a successful iterable traversal also returns a list; if traversal fails, the numerical comparison cannot succeed on a list, string, dictionary, or null. Thus in every profile:

> If a root opening returns a numeric result, its argument was numeric; if it succeeds on a list, its result is a list.

These statements concern finite successful evaluations. They assert neither arbitrary revalidation idempotence at ignored annotations nor unlimited-stack termination for the generic iterable profile.

7.3 No numeric goal can be proved from list-valued assumptions

Lemma. For any initial integer fresh-name floor and any finite remaining certificate, there is no finite successful invocation `c(a,g)` whose raw goal `a` is numeric and whose every context element is a list.

Proof. Suppose otherwise, and select such a successful invocation with the fewest calls to `c` in its completed recursive trace. Its first hypothesis check cannot close: the root opening of its numeric goal is numeric, and no numerical JSON value equals a Python list.

The invocation must therefore consume a command and successfully return from `c(m,g)`. If `m` were numeric, this would be a strictly shorter successful invocation of the forbidden kind. Thus the major is not numeric.

In either strict profile, a successfully checked nonnumeric major is a list. In the generic profile an empty string or an empty dictionary can also open to the empty list. A nonempty dictionary can occasionally have its iterable alias proved, but it cannot occur as a truthy major in the selected completed parent invocation: the subsequent expression `m * 3` raises a type error. A nonempty string has no finite successful opening. Consequently every truthy major that survives to the continuation exports a list, and every surviving false-valued major is an empty list or one of those harmless empty iterable aliases.

Consider the possible change of goal. If the matcher does not fire, the goal stays numeric. If it fires, its target test requires `i(t,u) == a`. The second root-opening fact therefore implies that `t` is numeric. By (Root scalar), `i(t,s)` and `i(t,u)` are the same root result. Both the Boolean-expression and the explicit-conditional implementations consequently retain a numeric goal. The Boolean implementation may retain the original raw zero alias when the transported result is false-valued; that makes no difference to this assertion.

If the major is truthy, the generated hypothesis is the list `m * 3`. If the major is false-valued and `u` is truthy, the computation expression constructs a two-element list. A successful opening of either hypothesis is a list. The final recursive call then has a numeric root-opened goal and an entirely list-valued context. It is a proper subtrace of the selected invocation, contradicting its minimality.

The only remaining case is introduction. Its expression `i(*a,t,2)` must unpack the numeric raw goal as an iterable, which raises a type error. That case cannot belong to a successful trace. All possibilities have been excluded. □

The minimization ranges over all starting floors and stacks. It is therefore unaffected by a recursively

checked major decreasing the global floor before its parent resumes. The argument uses finite completed traces, not a presumption that recursive checking always terminates.

7.4 Every reached command has a list-valued context

Initially the only assumption is the list `[]`. Suppose a command is reached with list-valued assumptions and its major check returns. The lemma excludes a numeric major. A truthy surviving major exports a list; a computation constructs a list. In introduction the assignment is

```
h = a = i(*a, t, 2)
```

in mathematical order. If its result is numeric, the later `len(a)` raises a type error before the continuation context is constructed, since this branch has false-valued `m`. Such a scalar matrix does *not* insert a scalar-zero assumption. If the assignment yields a list, successful opening again yields a list.

Both recursive premises thus receive list-valued contexts whenever execution reaches them. The expression `g + [new]` constructs a new list; a premise cannot mutate its caller's context. Induction on the reached calls proves the invariant. The tuple-based explanatory variant has the corresponding immutability property.

It follows that no numeric zero, Boolean `False`, floating zero, truthy numeric scalar, or newly allocated numeric name can be a successfully verified major at a reached command. In particular a closed scalar-zero goal is rejected by this interface. The canonical empty-array truth is accepted instead; mathematical completeness is claimed for canonical encodings, not for every redundant truth alias.

7.5 The unguarded matcher is safe on every reached continuation

If `s` is false-valued, both matcher alternatives are nonempty lists, so no false-valued major can match. If `s` is truthy, the second alternative is `False`. A false-valued major reaching this point is an empty list, or in the generic profile an empty string or dictionary. None equals `False`; a numeric major has already been excluded. Therefore, at every point reached after a successful major check, the matcher has exactly the same truth value as

```
m and m in ([s, u], not s and [t] * 3)
```

even though this identity is false on unrestricted Python tuples of values. No sentinel and no extra source guard are needed.

The proof depends on the initial context being `[[]]`, not `[0]`, and on premise contexts being freshly constructed. Silently changing either detail would invalidate this argument.

7.6 Why the relocated induction mode must be the initializer field

The corrected selector compares arity parity with `u`. In an introduction branch this field is necessarily false-valued. No false-valued JSON value compares equal to one. Thus a meaningful compound matrix, of odd arity one or three, cannot receive a progress assumption. A literal equation has even arity two and receives progress precisely for a zero-like mode such as `0`; an empty array mode chooses the zero instance. Surplus arities retain their inherited constant interpretations. Scalar matrices cannot survive the shape check discussed above.

Comparing instead with the unused `s` field would be wrong. That field is unrestricted in introduction, so choosing `s=1` would enable progress for odd-arity compound matrices. The distinction is semantic, not merely a preferred serialization.

In the computation branch `u` is truthy. Its new comparison can choose a fresh-name opening for some noncanonical scalar initializer aliases. This is handled by the inherited zero-instance discharge lemma, not by claiming that every generated open hypothesis is uniformly true. If the generated hypothesis is $H(\vec{b})$, its zero completion $H(\vec{0})$ is a computation theorem, while the old-floor validation makes the finite family of newly admitted names $\vec{b}$ fresh for the retained conclusion and context. Substituting zero for those names and cutting in the computation theorem discharges the extra hypothesis. Negative fractional names in the newly admitted interval are included separately in this finite substitution.

7.7 Conservativity and canonical completeness

The initial empty-array formula is already a theorem. For example the permitted nonempty-seed computation gives

$$C = 0, \qquad C := I(S(S0), S0; v.0).$$

Use this equation as both the equality major and the premise of transport through $F(w) \equiv (w = 0)$ to obtain $0 = 0$. Adding that fixed theorem to a proof context is conservative by substitution of its finite derivation for hypothesis uses. No new object-level truth predicate or logical axiom has been introduced.

For canonical certificates, substantive major commands retain their old fields. A computation uses the absent-major field `[]` and its existing nonempty initializer. An introduction uses absent major `[]`, with `u=0` for equation progress and `u=[]` for cases; its witness remains in `t`. The added empty major closes without a command or name allocation, so the remaining canonical proof simulation is unchanged. First applying the existing certified-zero repair supplies every unrestricted computation instance. This gives a finite certificate for every canonical derivation of the inherited native calculus.

Conversely, the invariant makes each reached matcher a legitimate guarded matcher. The remaining commands use the inherited transport, closure, paired-introduction/elimination, and zero-instance computation arguments, with the corrected equation-only mode test. Discharging the initial $0 = 0$ theorem yields a derivation in that same calculus. Its established whole-domain iPRA interpretations therefore transfer without changing the numerical domain, equality, or object language. The new result concerns a proof-checker presentation, not a new primitive-recursive arithmetic foundation.

7.8 Raw JSON types and the generic profile

The strict profiles recognize only Python lists as containers. The short predicate `a * 0 == []` has this effect for JSON values: lists pass; strings and numbers give different zero products; dictionaries and null raise. Its numerical fallback rejects strings. The root invariant covers all numbers and Booleans that reach it, including nonfinite values accepted by the default Python loader.

The generic profile has the inherited additional aliases. An empty string and an empty dictionary open to `[]`; a dictionary with only the empty-string key opens to `[[]]`, with its value ignored. A truthy dictionary cannot survive export as a major because `m * 3` raises. Nonempty strings and nonempty dictionary keys have no finite successful opening. Actual CPython rejects them through its recursion limit and subsequent type failure, while an ideal unlimited-stack execution can diverge. The raw soundness argument above covers every finite successful run of this profile; it does not assert ideal termination on every malformed finite JSON input.

Larger arrays remain arrays throughout opening, irrespective of their ignored annotations or depth increments. They therefore do not threaten the scalar-versus-list invariant. Their formula meanings and freshness behavior remain subject to the inherited sorted-decoder proof. Physical time, memory, and stack limits can reject sufficiently large genuine proofs in every profile; the completeness claim has the same adequate-resource, canonical-proof interpretation as its predecessors.

8 Why zero-source transport alone is insufficient

The native checker already shares the substitution path between equality transport and closure specialization from a zero instance. It is tempting to simplify the remaining match by allowing transport only from an atom $0 = t$. In the original presentation with computation leaves and no separately granted truth axiom, this loses even $0 = 0$. The counterinterpretation below does not apply verbatim to the new checker while retaining its initial truth context $[\,[]\,]$: that additional premise already supplies $0 = 0$ and is false in the counterinterpretation. Thus this obstruction does not rule out zero-source transport together with a separately primitive reflexivity or truth axiom.

Proposition 8.1. *In the initialized-iterator native calculus, replacing Tr by*

$$\frac{\Gamma \vdash 0 = t \quad \Gamma \vdash F(0)}{\Gamma \vdash F(t)}$$

does not suffice to derive $0 = 0$ *from the computation schemas and the remaining native rules.*

Proof. Give a proof-theoretic counterinterpretation on $\{0, 1\}$. Interpret $S0 = S1 = 1$, and interpret every iterator description as the constant function 1. Interpret the native atom $s = t$ as the relation

$$R(s, t) \quad \Longleftrightarrow \quad s = 1.$$

This auxiliary relation is *not actual equality*; we are proving nonderivability in a weakened calculus, not displaying an ordinary equality model of iPRA. Interpret Q, D by their ordinary formula meanings.

Every computation atom is true, since its left side is an iterator and therefore has value one. Every $0 = t$ atom is false, so the restricted transport rule is vacuously sound. Every domain element is zero or a successor. Moreover induction is valid for every predicate on this domain: its base gives the value at zero and its successor step at zero gives the value at one. Thus DI-case and even unrestricted DI-progress are sound in this auxiliary interpretation. Hyp, QE, DE, and structural rules are sound as well. Induction on finite derivations preserves truth. But $R(0, 0)$ is false. Hence no derivation of $0 = 0$ exists. □

Unrestricted transport does derive it. Take the computation $H : I(0, 0; v.0) = 0$ and the motive $F(w) \equiv w = 0$. Use H as both the major equality and the proof of $F(I(0, 0; v.0))$. Tr gives $F(0)$. If the checker restricts literal-zero initializers, first obtain H by the certified-zero repair. The issue is the transport restriction, not that optional direct-computation restriction.

The accompanying finite program checks the truth-table portions of this counterinterpretation. The mathematical argument above, rather than a bounded search for proofs, establishes nonderivability. It rules out this particular fusion; it is not a lower bound on all iPRA checkers or on all possible recodings of equality.

9 Further research questions

1. **A measured checker for the new numerical basis.** Can the tied-seed and monus computation schemas be checked more simply than arbitrary initialized scalar iteration, once every logical rule, scope check, parser, and driver is charged? The present small checker does not answer this question: it intentionally retains the inherited I language.

2. **One program below all coordinates.** Can a structurally terminating explicit-parser implementation beat the new family envelope in a single source file? The generic and byte profiles should remain separate observations. Gzip size should remain informational rather than the objective.

3. **Mechanical verification of the bootstrap.** The most useful immediate formalization target is the gated-complement identity with its fixed parameter, followed by the native contextual-case lemma using H. The bounded compiler's cap coherence and finite-case induction preservation should then be verified, with actual source and target syntax and both return maps.

4. **The minimum-only candidate.** Replacing the equality primitive by three recursive minimum equations was investigated. The term $\mathrm{parity}(\min(2x, 2y+1))$ has the intended comparator meaning in the standard model. Obtaining all needed monus equations and whole-domain reflection from that basis, without shifted-parameter induction, remains open here. The positive theorem in this report uses the explicitly stated monus equations instead.

5. **General neutral-premise transformations.** The checker reduction suggests a general method: adjoin a derivable neutral premise so a partial proof interface becomes total. Determine which other native rule interfaces admit this transformation without adding parser complexity or allowing malformed values to imitate a genuine inference. The numeric-major exclusion and the false-valued mode field are substantive obligations in the present example.

6. **Proof length and readability.** The small trusted source should also be compared with certificate size, maximum call depth, compilation cost, and the complexity of its soundness proof. The neutral-premise transformation adds a proof call that closes immediately, but does not require a new serialized subproof for its neutral major. Triplicated closure matrices remain a separate source of certificate growth.

10 Reproduction and trust boundary

Run `python3 verify.py` from the extracted directory to reproduce the source measurements, positive and negative certificates in ordinary and optimized Python, finite arithmetic diagnostics, and transport counterinterpretation. Run `python3 build.py` to compile this report. The recorded runtime and results are in `evidence/verification.json`.

The complete theorem checkers import only the standard JSON library. Producers, the readable expansion, diagnostics, measurement scripts, and build tools are outside their trusted source. No input Python expression is evaluated. Acceptance requires normal termination and exactly `ACCEPT` on stdout; an exception or nonzero exit is rejection. Unused certificate prefixes are inert, matching the incumbent interface.

Finite tests support implementation fidelity; the written invariants, derivation translations, and interpretation proofs carry the general claims. Physical stack and memory limits restrict every executable implementation; canonical proof completeness concerns its unbounded-resource mathematical abstraction. The generic traversal has the additional malformed-string qualification already disclosed.

A Attributed bounded initialization and finite function graphs

This appendix reproduces the bounded-initializer draft's construction [3], beginning after its equality characteristic has become available. It is included so the interpretation claim is inspectable without retrieving another archive. The inherited monus term is written $\widehat{d}$ to distinguish it from our primitive d; numerical constant-body recursion is C rather than D. The local hit function H below is unrelated to the bounded selector called H in the native bootstrap. All other mathematical content is retained, with notation and cross-reference repairs. The hypotheses about ϵ are supplied by our proved reflection theorem, rather than new axioms of T_{Bd}.

A.1 Order and bounded arithmetic

For a Boolean b, let $J(a,b) := B(Zb; u, v.a)$, so $J(a,0) = a$ and $J(a,1) = Sa$. Define

$$\begin{aligned} H(n,y) &:= L(n; r, v.J(v, \epsilon(r,y))),\\ \widehat{d}(x,y) &:= L(x; u, v.J(v, H(Su,y))),\\ \mathrm{mn}(x,y) &:= \widehat{d}(x, \widehat{d}(x,y)),\\ \mathrm{db}(n) &:= L(n; u, v.S(Sv)),\\ \mathrm{ad}_b(a,c) &:= \widehat{d}\big(\widehat{d}(\mathrm{db}(b), \widehat{d}(b,a)), \widehat{d}(b,c)\big). \end{aligned}$$

All five are terms in the original finite-description language. Use $x \le y$ as the quantifier-free abbreviation $\widehat{d}(x,y) = 0$. The elementary order lemma proves that this is a discrete linear order, that $\widehat{d}$ is its truncated difference, and that mn is its minimum. It also proves, entirely by quantifier-free induction, the following bounded-addition identities:

$$a \le b \implies \mathrm{ad}_b(a,0) = a, \tag{5}$$

$$a, Sc \le b \implies \mathrm{ad}_b(a,Sc) = S\,\mathrm{ad}_b(a,c), \tag{6}$$

$$a, c \le b \implies \mathrm{ad}_b(a,c) \le \mathrm{db}(b), \tag{7}$$

$$\mathrm{ad}_b(b,b) = \mathrm{db}(b). \tag{8}$$

The function $c \mapsto \mathrm{ad}_b(a,c)$ is nondecreasing on $c \le b$, when $a \le b$. Only these bounded identities are used; there is no uncharged total binary addition term in the source language. The detailed inductions are given in Appendix E.

Define

$$\mathrm{sel}_b(c,a,e) := \mathrm{ad}_b\big(G(Zc,a), G(c,e)\big).$$

If $a, e \le b$, coverage and the gate equations show that this returns a at $c = 0$ and e at every successor control. At most one summand is nonzero, so the result is again at most b.

For any fixed earlier update h, define the clipped initialized recursor

$$R_b(n,a; u, v.h) := L\big(Sn; r, s.\,\mathrm{sel}_b(r, \mathrm{mn}(a,b), \mathrm{mn}(h[Pr/u, s/v], b))\big). \qquad \text{(Clipped recursion)}$$

The gate, minimum, and L equations give

$$R_b(0,a;h) = \mathrm{mn}(a,b), \tag{9}$$

$$R_b(Sn,a;h) = \mathrm{mn}\big(h(n, R_b(n,a;h)), b\big). \tag{10}$$

Quantifier-free induction gives $R_b(n,a;h) \le b$ for every counter. The cap b is a passive parameter throughout the recursion. This supplies arbitrary initialization whenever an independent cap has first been fixed. It is compatible with the one-coordinate growth obstruction: the output is bounded in the cap coordinate.

B A compiler with internal bounds and cap coherence

Fix the usual finite primitive-recursive descriptions built from zero, successor, projections, composition, and primitive recursion. The following theorem is an external construction on descriptions, producing finite source terms and finite source proofs. It does not add a uniform evaluation function for arbitrary description codes.

Theorem B.1 (Internal bounded realization)**.** *For every fixed k-ary primitive-recursive description f, one can construct source terms*

$$\tau_f(b, x_1, \dots, x_k), \qquad M_f(b)$$

and proofs in $T_{B\epsilon}$ with the following properties.

1. *M_f is nondecreasing and $b \le M_f(b)$.*
2. *The bound $\tau_f(b, \vec{x}) \le M_f(b)$ holds for all inputs, whether or not the inputs are bounded by b.*
3. *If every $x_i \le b$ and every $x_i \le c$, then $\tau_f(b, \vec{x}) = \tau_f(c, \vec{x})$.*
4. *The finite-case graphs built from these terms satisfy the defining equations of the description f, as proved below.*

All comparisons are the quantifier-free source abbreviations just defined. The construction produces proofs on the whole domain, not just correctness on standard numerals.

Proof. We construct the first three items by induction on the fixed description. The fourth will then follow from the actual defining formulas.

Initial descriptions. For zero use $\tau_0 = 0$; for a projection use $\tau_{\pi_i}(b, \vec{x}) = \mathrm{mn}(x_i, b)$; for successor use $\tau_S(b, x) = S\,\mathrm{mn}(x, b)$. The common bound $M(b) = Sb$ works. The minimum identities prove bounds and coherence immediately.

Composition. Suppose $f(\vec{x}) = h(g_1(\vec{x}), \dots, g_m(\vec{x}))$. Write $G_i = M_{g_i}$ and $H = M_h$. Put

$$Q = G_1 \circ \cdots \circ G_m, \qquad M_f = H \circ Q,$$

with Q the identity if $m = 0$. Since all G_i are nondecreasing and inflationary, $G_i(b) \le Q(b)$ for each i. Define

$$\tau_f(b, \vec{x}) = \tau_h\big(Q(b), \tau_{g_1}(b, \vec{x}), \dots, \tau_{g_m}(b, \vec{x})\big).$$

The global bound follows from the bound for τ_h. For coherence, the inner values agree at any two input caps by their induction hypotheses. They are bounded by both $Q(b)$ and $Q(c)$; applying coherence for h proves the result. Monotonicity and inflationarity of M_f follow by composition.

Primitive recursion and its majorant. Suppose

$$f(0, \vec{x}) = g(\vec{x}), \qquad f(Sn, \vec{x}) = h(n, f(n, \vec{x}), \vec{x}).$$

Let $G = M_g$, $H = M_h$, and put

$$F = H \circ G \circ S.$$

This is a source unary term with internal proofs of monotonicity and $Sz \le F(z)$. Let

$$I_F(t) := L(t; u, v.F(v)), \qquad C_b := I_F(S\,\mathrm{db}(b)), \qquad M_f(b) := C_b. \tag{Majorant}$$

Equation induction gives $I_F(0) = 0$, $I_F(St) = F(I_F(t))$, and $t \le I_F(t)$. Its monotonicity is a quantifier-free induction using order discreteness: if $s \le St$, either $s = St$ or $s \le t$. Consequently $b \mapsto C_b$ is nondecreasing and inflationary. This construction uses only successor, composition, and zero-start iteration for its majorants.

For $j \le b$, use the following actual source term for the stage bound:

$$K_b(j) := I_F(S\,\mathrm{ad}_b(b, j)). \tag{Stage bound}$$

The bounded-addition identities prove

$$b, G(b) \le K_b(0), \tag{11}$$
$$Sj \le b \implies K_b(Sj) = F(K_b(j)), \tag{12}$$
$$j \le b \implies b \le K_b(j) \le C_b. \tag{13}$$

For example, $K_b(0) = I_F(Sb) = F(I_F(b)) \ge G(b)$ because $I_F(b) \ge b$, G is nondecreasing, and H is inflationary. The upper bound uses $\mathrm{ad}_b(b, j) \le \mathrm{db}(b)$. Also $H(z) \le F(z)$, by inflationarity of G and monotonicity of H.

Write $n_b = \mathrm{mn}(n, b)$ and $\vec{x}_b = (\mathrm{mn}(x_i, b))_i$. Define

$$\tau_f(b, n, \vec{x}) := R_{C_b}(n_b, \tau_g(b, \vec{x}_b); u, v.\tau_h(C_b, u, v, \vec{x}_b)). \tag{Compiled recursion}$$

Its global bound is immediate from the clipped-recursion bound.

No clipping on bounded inputs. Fix passive parameters $\vec{x} \le b$ and let

$$r_b(j) := R_{C_b}(j, \tau_g(b, \vec{x}); u, v.\tau_h(C_b, u, v, \vec{x})).$$

We prove by quantifier-free induction on j that

$$j \le b \longrightarrow r_b(j) \le K_b(j). \tag{Invariant}$$

At zero, $\tau_g(b, \vec{x}) \le G(b) \le K_b(0) \le C_b$, so the seed is not clipped. For the step assume $Sj \le b$. Then the induction hypothesis applies at j; all arguments $j, r_b(j), \vec{x}$ of the step function are bounded by $K_b(j)$, as well as by C_b. Coherence for the already constructed description h gives

$$\tau_h(C_b, j, r_b(j), \vec{x}) = \tau_h(K_b(j), j, r_b(j), \vec{x}) \le H(K_b(j)) \le F(K_b(j)) = K_b(Sj) \le C_b.$$

Thus its value is not clipped and the invariant follows. The same argument establishes the actual untruncated step identity

$$Sj \le b \implies r_b(Sj) = \tau_h(C_b, j, r_b(j), \vec{x}). \tag{Unclipped step}$$

The matrix of the induction is quantifier free, and the input cap and all passive parameters remain fixed throughout it.

Coherence between two arbitrary caps. Fix b, c and inputs $n, \vec{x}$ bounded by both. After the minimum identities remove their input clipping, compare the two runs r_b, r_c. Quantifier-free induction on j proves

$$(j \le b \wedge j \le c) \longrightarrow r_b(j) = r_c(j). \tag{Two-cap invariant}$$

The base uses coherence for g and the already proved absence of seed clipping. If $Sj \le b, c$, the two states at j are equal by the induction hypothesis and bounded by both C_b, C_c. The other

step arguments are bounded by both caps too. Coherence for h equates the two raw step values. Formula (Unclipped step) removes their clipping, giving the successor equality. At $j = n$ this is exactly the required coherence of τ_f. No induction on a universally quantified formula has occurred: $b, c, n, \vec{x}$ are fixed parameters, and both displayed invariant matrices are quantifier free.

Every recursive step body in this construction is the already compiled original-language term τ_h, inlined before it is inserted into L or R_b. No recursion schema is extended for free to an opaque function adjoined later by a graph definition. This distinction prevents a conservative-definition argument from hiding a stronger recursion principle.

This completes the construction of the terms and internal proofs. □

C Function graphs and preservation of induction

For a nonempty tuple $\vec{x} = (x_1, \dots, x_k)$ let

$$C_i(\vec{x}) := \bigwedge_{j<i} x_j < x_i \ \wedge \ \bigwedge_{j>i} x_j \leq x_i.$$

This says that coordinate i is the first largest one. Discrete linearity proves that these fixed finitely many quantifier-free cases are exhaustive and disjoint. Define

$$\Phi_f(\vec{x}, z) := \bigvee_{i=1}^{k} \big(C_i(\vec{x}) \wedge z = \tau_f(x_i, \vec{x})\big). \tag{Graph}$$

For a nullary description use $z = \tau_f(0)$.

A concrete example: binary addition. The bounded-addition term already illustrates the method without the full compiler. Its total single-valued graph is

$$\Phi_+(x, y, z) := \big(y \leq x \wedge z = \mathrm{ad}_x(x, y)\big) \ \vee \ \big(x < y \wedge z = \mathrm{ad}_y(x, y)\big).$$

On either case the selected cap bounds both arguments. The bounded-addition identities and coherence under changing a valid cap give the usual addition equations. Each possible output term is bounded by twice one input, while addition itself is not bounded by any unary function of one input. Thus graph interpretation succeeds exactly where a single-term definition is obstructed.

Lemma C.1 (Totality, uniqueness, and computation)**.** *The theory proves that every Φ_f is total and single-valued. In the definitional extension by its uniquely specified function f^*, the defining composition and primitive-recursion equations of the description f hold. The extension admits quantifier-free induction for formulas containing finitely many of these defined functions.*

Proof. Exhaustiveness provides a displayed value term on each case, proving existence. Disjointness proves uniqueness. Moreover, for every source cap b bounding the arguments, coherence proves

$$f^*(\vec{x}) = \tau_f(b, \vec{x}). \tag{Cap realization}$$

One proves this by the finite graph cases: on case i, both b and x_i bound all arguments, so the corresponding values are equal.

Initial equations follow from the compiled initial descriptions. For composition, use any cap b bounding the common input tuple. The compiled inner values are bounded by $Q(b)$, so applying (Cap realization) at $Q(b)$ to the outer function proves the composition equation directly from its compiler definition.

For recursion at counter zero, choose a graph case whose cap bounds $0, \vec{x}$. The seed bound prevents clipping, so the value is $\tau_g(b, \vec{x}) = g^*(\vec{x})$. At a successor counter, choose a case whose cap b bounds $Sn, \vec{x}$. Then $n \leq b$, the unclipped-step identity applies, and (Cap realization) identifies the previous state with $f^*(n, \vec{x})$. All step arguments are bounded by C_b; the same cap realization for h identifies the update with $h^*(n, f^*(n, \vec{x}), \vec{x})$. This proves the recursion equations.

It remains to justify the induction claim, rather than merely invoke conservativity of definitional extensions. Every occurrence of a defined function has a graph given by finitely many quantifier-free cases and explicit source value terms. By external induction on a finite term, it therefore has a finite family of pairs (A_j, t_j), with source quantifier-free guards A_j and source terms t_j, such that the guards cover all inputs and the term equals t_j on guard A_j. At a composition of terms take the finite Cartesian product of the argument cases and then apply the finitely many cases of the outer function. This introduces no object-language existential quantifier. Guards need not be disjoint after expansion; their associated values agree on every overlap.

For a quantifier-free formula involving defined functions, take a common finite refinement of the cases of all its terms. On each guard replace its terms by their source values and retain its actual intuitionistic connectives. The finite disjunction of these guarded source formulas is quantifier free and is provably equivalent to the original formula. Covering cases justify the equivalence also for implication and negation. Applying source quantifier-free induction to this equivalent formula transfers induction to the expanded formula. This is the essential finite-case step: an arbitrary existential graph translation would not suffice. □

D Whole-domain bi-interpretability

Theorem D.1 (Arithmetic equivalence)**.** *The explicitly axiomatized theory $T_{B\epsilon}$ and full first-order intuitionistic primitive recursive arithmetic are bi-interpretable. Both interpretations use the whole numerical domain and actual equality. Their composite definable isomorphisms are identity maps. In particular the theories are equiconsistent, and neither an appeal to standard-model agreement nor induction on quantified matrices is needed for this conclusion.*

Proof. **Interpretation in iPRA.** Interpret $0, S$ ordinarily and ϵ as iPRA's primitive-recursive equality characteristic. Translate every fixed staged B_h symbol by primitive recursion with initial value $Sh(0, 0, \vec{p})$ and update $h(n, v, \vec{p})$, first translating its earlier-stage body. The two computation equations become defining primitive-recursion equations. The equality-test, separation, and contextual equation-induction principles hold in iPRA. Logical connectives, quantifiers, and genuine falsity retain their ordinary intuitionistic meanings.

Interpretation of iPRA in the source. Interpret each fixed primitive-recursive function description by its total unique graph Φ_f and use ordinary relational interpretation of function terms, or equivalently work in the conservative definitional extension by the functions f^*. The graph computation lemma proves every defining equation. The source predecessor and separation prove the successor boundary laws. The finite-case quantifier-free elimination in that lemma proves every translated iPRA induction instance. Thus all axioms and inference rules of full iPRA are interpreted, with their intuitionistic logic unchanged.

The composite on the iPRA side. An externally fixed primitive-recursive description is uniquely determined by its initial and step equations. External induction on its finite description, with ordinary quantifier-free induction on the recursion counter at each recursion node, proves in iPRA that its

composite interpretation equals the original function. The necessary graph functions can be treated in their conservative definitional extensions; finite-case elimination supplies any needed induction instances. Hence the identity on the domain preserves every primitive-recursive function and actual equality.

The composite on the source side. Proceed by external induction on the finite stage of a source description. The ordinary $0, S$ operations are preserved. The interpreted equality characteristic and the original ϵ are both Boolean characteristics of actual equality; their range and reflection properties prove that they agree by the two Boolean cases. For a fixed B_h, the earlier-stage induction hypothesis identifies the interpreted update with its original source body h. Both the original function and its composite satisfy (B0) and (BS) with that same body. Induction on the equation asserting their equality proves recurrence uniqueness and identifies them at every counter and every passive parameter. If the composite is expressed by defined graph functions, the finite-case elimination lemma again legitimizes this induction in the original theory. Thus the identity map preserves every source function symbol too.

Since all numerical operations and equality are preserved, external induction on arbitrary formula syntax proves preservation of all logical formulas under both composites. Quantifiers range over the same whole domain. This establishes the two identity definable isomorphisms and the claimed full bi-interpretation. Genuine falsity is preserved in both directions, giving equiconsistency. □

E Internal order, difference, and addition under a bound

This attributed appendix supplies the arithmetic facts needed by the bounded compiler [3]. The notation $\widehat{d}$ distinguishes its constructed difference from the primitive d of our new basis. All its inductions are on quantifier-free formulas with the other displayed variables held fixed as parameters. In particular, an induction below is never shorthand for induction on a universally quantified formula. Use ϵ for the numerical equality test, with values zero and one, and use the already derived terms and equations

$$P0 = 0, \quad P(Sx) = x, \qquad Z0 = 1, \quad Z(Sx) = 0, \qquad G(0, a) = 0, \quad G(Sx, a) = a,$$

$$L_h(0) = 0, \quad L_h(Sn) = h(n, L_h(n)), \qquad J(a, 0) = a, \quad J(a, 1) = Sa.$$

Here $1 = S0$. Applying P to an equality of successors gives successor injectivity. Applying Z to $0 = Sx$ contradicts $0 \neq 1$, so zero is distinct from every successor. The proved Boolean range and reflection laws consequently give

$$\epsilon(x, x) = 1, \quad \epsilon(x, y) = \epsilon(y, x), \quad \epsilon(Sx, Sy) = \epsilon(x, y), \quad \epsilon(0, Sx) = 0.$$

Each identity follows by the two possible Boolean values and the reflection law for ϵ; these are derived identities, not extra axioms.

Lemma E.1 (Derived constructor coverage)**.** *The theory proves* $x = 0 \vee x = S(Px)$.

Proof. Apply the already derived quantifier-free induction to $x = 0 \vee x = S(Px)$. The base is immediate. At a successor the second disjunct follows directly from $P(Sx) = x$, independently of the induction hypothesis. Thus constructor coverage need not be a primitive axiom. □

E.1 A hit term defines a discrete linear order

Define

$$H(n,y) := L(n; r, v.J(v, \epsilon(r,y))).$$

Its defining equations are

$$H(0,y) = 0, \qquad H(Sn,y) = J(H(n,y), \epsilon(n,y)). \tag{H}$$

Lemma E.2 (Hit identities and duality)**.** *The following identities and range assertion hold internally:*

$$\begin{aligned} &H(Sn,0) = 1, && H(Sn,Sy) = H(n,y), && H(n,n) = 0,\\ &H(Sn,n) = 1, && H(n,y) = 0 \vee H(n,y) = 1, \\ &H(Sy,x) = Z(H(x,y)). \end{aligned}$$

Moreover a hit persists: $H(n,y) = 1$ *implies* $H(Sn,y) = 1$*.*

Proof. Equation induction on n proves $H(Sn,0) = 1$: its first step hits zero, and every later equality test against zero is zero. Equation induction on n, with y fixed, proves $H(Sn,Sy) = H(n,y)$, using $\epsilon(Sn,Sy) = \epsilon(n,y)$. A further equation induction gives $H(n,n) = 0$; the recurrence then gives $H(Sn,n) = 1$.

For Boolean range use quantifier-free induction on n, with y fixed. If $\epsilon(n,y) = 0$, the state is unchanged. If $\epsilon(n,y) = 1$, reflection gives $n = y$, so the old state is $H(n,n) = 0$ and the new state is one. If $H(n,y) = 1$, diagonal zero shows $n \neq y$, so the next equality test vanishes. This proves persistence. The same case split shows

$$H(Sn,y) = 1 \implies H(n,y) = 1 \ \vee \ n = y. \tag{New hit}$$

Finally prove duality by equation induction on x, with y fixed. At zero both sides are one. In the successor step the new left side is $H(Sy,Sx) = H(y,x)$, whereas the old left side is $J(H(y,x), \epsilon(y,x))$. If $\epsilon(x,y) = 0$, both relevant recurrences leave their states unchanged, and the induction hypothesis gives the required equality. If $\epsilon(x,y) = 1$, put $x = y$; the new left side is zero and the new right side is $Z(H(Sx,x)) = Z1 = 0$. □

Put

$$x < y \quad :\iff \quad H(y,x) = 1, \qquad x \leq y \quad :\iff \quad H(Sy,x) = 1. \tag{Order}$$

These are quantifier-free abbreviations.

Lemma E.3 (Discrete linear-order laws)**.** *The relations in* (Order) *form a discrete linear order with least element zero. In particular,*

$$x \leq y \iff (x = y \vee x < y), \qquad x < Sy \iff x \leq y,$$

successor preserves and reflects both comparisons, and $x \leq Sy$ *with* $x \neq Sy$ *implies* $x \leq y$*.*

Proof. Diagonal identities give reflexivity of $\leq$, irreflexivity of $<$, and $x < Sx$. The recurrence for $H(Sy,x)$, split on $\epsilon(y,x)$, gives the first displayed equivalence. Duality and Boolean range give trichotomy: if $x \neq y$, then $H(Sy,x) = H(y,x)$, and $H(x,y)$ is its Boolean complement. Thus exactly one of $x < y$, $x = y$, $y < x$ holds.

For strict transitivity retain $H(y,x)=1$ and induct on z in the quantifier-free formula

$$H(z,y)=1 \;\longrightarrow\; H(z,x)=1.$$

The base has false antecedent. At the successor, the new-hit implication (New hit) gives either $H(z,y)=1$, when the induction hypothesis followed by persistence applies, or $z=y$, when the retained $H(y,x)=1$ followed by persistence applies. Transitivity of $\leq$ and the mixed comparison laws now follow by splitting equality from strict comparison. Trichotomy gives antisymmetry.

The identities $H(Sn,Sy)=H(n,y)$ give successor preservation and reflection. The equivalence $x<Sy \leftrightarrow x\leq y$ is immediate from the definitions, and the asserted discrete predecessor step follows from it and the first equivalence. Finally $H(Sx,0)=1$ gives $0\leq x$; if $x\leq 0$, trichotomy or constructor coverage gives $x=0$. □

E.2 Difference is an internal term, not an imported operation

Define

$$\widehat{d}(x,y) := L(x;u,v.J(v,H(Su,y))).$$

Thus

$$\widehat{d}(0,y)=0, \qquad \widehat{d}(Sx,y)=J(\widehat{d}(x,y),H(Sx,y)). \tag{d}$$

Lemma E.4 (Difference laws)**.** *The theory proves*

$$\begin{array}{lll} \widehat{d}(x,0)=x, & \widehat{d}(Sx,Sy)=\widehat{d}(x,y), & \widehat{d}(x,x)=0, \\ \widehat{d}(x,y)=0 \iff x\leq y, & \widehat{d}(x,Sy)=P(\widehat{d}(x,y)), & \widehat{d}(x,y)\leq x. \end{array}$$

Difference is monotone in its first argument and antitone in its second. Furthermore

$$\begin{aligned} y\leq x &\implies \widehat{d}(Sx,y)=S(\widehat{d}(x,y)), \\ 0<y\leq x &\implies \widehat{d}(x,Py)=S(\widehat{d}(x,y)). \end{aligned}$$

Proof. Equation induction gives $\widehat{d}(x,0)=x$ because $H(Sx,0)=1$. For simultaneous successors, induct on x with y fixed. The base uses $H(S0,Sy)=0$. At the successor use $H(S(Sx),Sy)=H(Sx,y)$ and the induction hypothesis in (d).

Quantifier-free induction on x, with y fixed, proves $\widehat{d}(x,y)=0 \leftrightarrow H(x,y)=0$. At the successor, if $H(Sx,y)=1$, the new difference is a successor and is nonzero. If $H(Sx,y)=0$, persistence implies $H(x,y)=0$, so the old difference is zero and remains zero. Duality identifies $H(x,y)=0$ with $H(Sy,x)=1$, proving the order characterization and $\widehat{d}(x,x)=0$.

Now induct on x for $\widehat{d}(x,Sy)=P(\widehat{d}(x,y))$, with y fixed. Its successor left side is $\widehat{d}(x,y)$. The right side is $P(J(\widehat{d}(x,y),H(Sx,y)))$. When the Boolean is one, predecessor cancels the successor; when it is zero, the preceding zero characterization gives $\widehat{d}(x,y)=0$. Both cases give the desired equality.

Quantifier-free induction on x proves $\widehat{d}(x,y)\leq x$: its successor state is either the old state or its successor. For first-argument monotonicity induct on v in

$$u\leq v \;\longrightarrow\; \widehat{d}(u,z)\leq \widehat{d}(v,z),$$

with u,z fixed. At Sv, either $u=Sv$ or discreteness gives $u\leq v$; in the latter case use the induction hypothesis and the fact that the new state is at least its old state. The base uses $u\leq 0\to u=0$.

For second-argument antitonicity retain $y \le z$ and induct on x in $\widehat{d}(x,z) \le \widehat{d}(x,y)$. If $H(Sx,z) = 0$, the left state stays fixed and the right state cannot decrease. If $H(Sx,z) = 1$, then $z \le x$; transitivity gives $y \le x$, so both states increment and successor preserves their old comparison.

The first final implication is the defining recurrence, since $y \le x$ means $H(Sx,y) = 1$. For the second, coverage gives $y = S(Py)$ when $y > 0$. The inequalities imply $Py < x$, so $\widehat{d}(x,Py) \neq 0$. Functional coverage at this positive difference and $\widehat{d}(x,S(Py)) = P(\widehat{d}(x,Py))$ yield $\widehat{d}(x,Py) = S(\widehat{d}(x,y))$. □

Define

$$\mathrm{mn}(x,y) := \widehat{d}(x,\widehat{d}(x,y)).$$

Lemma E.5 (Minimum)**.** *If $x \le y$, then $\mathrm{mn}(x,y) = x$; if $y \le x$, then $\mathrm{mn}(x,y) = y$. Consequently* mn *is the minimum operation for the internal linear order.*

Proof. The first claim uses $\widehat{d}(x,y) = 0$ and $\widehat{d}(x,0) = x$. For the second induct on y in $y \le x \to \widehat{d}(x,\widehat{d}(x,y)) = y$, with x fixed. The base is $\widehat{d}(x,x) = 0$. If $Sy \le x$, then $y < x$ and $0 < \widehat{d}(x,y) \le x$. Apply the final difference implication to obtain

$$\widehat{d}(x,\widehat{d}(x,Sy)) = \widehat{d}(x,P(\widehat{d}(x,y))) = S(\widehat{d}(x,\widehat{d}(x,y))) = Sy.$$

The last equality uses the induction hypothesis, which applies because $y \le x$. Linearity completes the interpretation of mn. □

E.3 Addition with an explicit common bound

Define the unary doubling term

$$\mathrm{db}(n) := L(n;u,v.S(Sv)).$$

Thus $\mathrm{db}(0) = 0$ and $\mathrm{db}(Sn) = S(S(\mathrm{db}(n)))$. The notation $2n$ in the compiler may be read as this term $\mathrm{db}(n)$; it does not invoke binary addition.

Lemma E.6 (Doubling identities)**.** *The term* db *is monotone, $n \le \mathrm{db}(n)$, and $\widehat{d}(\mathrm{db}(n),n) = n$.*

Proof. Quantifier-free induction gives $n \le \mathrm{db}(n)$ directly from the two successors in the defining equation. Monotonicity follows by induction on the larger argument, splitting equality at its successor from the discrete predecessor case, just as for first-argument monotonicity of d. Equation induction proves the remaining identity. At the successor,

$$\widehat{d}(\mathrm{db}(Sn),Sn) = \widehat{d}(S(S(\mathrm{db}(n))),Sn) = \widehat{d}(S(\mathrm{db}(n)),n) = S(\widehat{d}(\mathrm{db}(n),n)) = Sn,$$

where the third equality uses $n \le \mathrm{db}(n)$. □

Put

$$K_b(a) := \widehat{d}(\mathrm{db}(b),\widehat{d}(b,a)), \qquad \mathrm{ad}_b(a,c) := \widehat{d}(K_b(a),\widehat{d}(b,c)). \qquad \text{(Bounded addition)}$$

Every expression here is a finite term of the original language.

Lemma E.7 (Bounded-addition laws)**.** *Internally, the following hold:*

1. *If $a \le b$, then $\mathrm{ad}_b(a,0) = a$.*
2. *If $a \le b$ and $Sc \le b$, then $\mathrm{ad}_b(a,Sc) = S(\mathrm{ad}_b(a,c))$.*
3. *If $c \le b$, then $\mathrm{ad}_b(0,c) = c$.*

4. $\mathrm{ad}_b(b,b) = \mathrm{db}(b)$ *and* $\mathrm{ad}_b(a,c) \le \mathrm{db}(b)$ *for all* a, c.

5. ad_b *is monotone in its second argument.*

6. *If* $a, c \le b$ *and* $a, c \le b'$, *then* $\mathrm{ad}_b(a,c) = \mathrm{ad}_{b'}(a,c)$.

Thus it supplies addition on inputs bounded by b, *without asserting an unrestricted binary addition term.*

Proof. Since $\widehat{d}(b,a) \le b$, antitonicity and doubling give

$$b = \widehat{d}(\mathrm{db}(b), b) \le K_b(a) \le \mathrm{db}(b). \qquad \text{(K bounds)}$$

Also $K_b(0) = b$. If $Sa \le b$, put $q = \widehat{d}(b,a)$. Then $0 < q \le b \le \mathrm{db}(b)$ and $\widehat{d}(b, Sa) = Pq$, so

$$K_b(Sa) = \widehat{d}(\mathrm{db}(b), Pq) = S(K_b(a)).$$

Quantifier-free induction on a now proves the first assertion under $a \le b$. The base is $\widehat{d}(b,b) = 0$; at the successor use the last identity and $b \le K_b(a)$ to increment $\widehat{d}(K_b(a), b)$.

For the second assertion put $q = \widehat{d}(b,c)$. The premise $Sc \le b$ gives $0 < q \le b \le K_b(a)$. Hence

$$\mathrm{ad}_b(a, Sc) = \widehat{d}(K_b(a), Pq) = S(\widehat{d}(K_b(a), q)) = S(\mathrm{ad}_b(a,c)).$$

The third assertion follows by quantifier-free induction on $c \le b$ using the first two. For the fourth, substitute $a = c = b$ and use $\widehat{d}(b,b) = 0$ twice; the upper bound follows from $\widehat{d}(K_b(a), \widehat{d}(b,c)) \le K_b(a) \le \mathrm{db}(b)$.

If $c \le c'$, antitonicity gives $\widehat{d}(b,c') \le \widehat{d}(b,c)$; applying antitonicity again to the outer difference gives $\mathrm{ad}_b(a,c) \le \mathrm{ad}_b(a,c')$. Finally retain $a \le b, b'$ and induct on c in the quantifier-free implication

$$c \le \mathrm{mn}(b,b') \;\longrightarrow\; \mathrm{ad}_b(a,c) = \mathrm{ad}_{b'}(a,c).$$

The base gives a on each side, and whenever $Sc \le \mathrm{mn}(b,b')$ the successor laws apply to both sides and preserve the induction hypothesis. □

These proofs establish the numerical prerequisites for bounded selection and clipped recursion entirely within the proposed theory. Each use of a case split is justified by numerical equality decidability, Boolean range, or the derived linear-order and coverage laws. No standard-model correctness assertion and no hidden unrestricted addition operation is used as an internal axiom.

Corollary E.8 (Identification with the new primitive monus)**.** *In* T_{Bd}, $\widehat{d}(x,y) = d(x,y)$ *for every* x, y.

Proof. Keep x fixed and apply literal-equation induction in y. Both terms have value x at zero. Their successor values are the predecessors of their previous values, by the proved difference lemma and the primitive monus equation. Congruence of P proves the step. All occurrences of ϵ in $\widehat{d}$ are inlined as the fixed $B + d$ term from the main text. Thus this is a source-language equation induction, without an appeal to a graph-function extension. □

F Complete measured source

The primary implementation uses an explicit list test. The byte profile changes only that test to the inherited exact-JSON predicate `a*0 == []`. The generic profile has the runtime qualification stated in the implementation section. Each is a complete standalone program, including JSON input and the acceptance driver.

F.1 Explicit list test

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <=
     a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g)
    a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t,
    2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)])
q = json.load(open(0)).pop
c(q(), [[]])
print('ACCEPT')
```

F.2 Byte profile

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d
    else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g)
    a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t,
    2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)])
q = json.load(open(0)).pop
c(q(), [[]])
print('ACCEPT')
```

F.3 Generic traversal profile

```
from json import load
k = 0

def i(a, t=None, d=0):
    try:
        return [i(b, t, d + 5 - len(a)) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g)
    if m in ([s, u], not s and [t] * 3) and i(t, u) == a:
        a = i(t, s)
    h = m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t,
    2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)])
q = load(open(0)).pop
c(q(), [[]])
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *ProveIt*, `docs/drafts`, commit `76a2651601d17b34d46753f189ea026a21ce8d29`, inspected 10 November 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/76a2651601d17b34d46753f189ea026a21ce8d29/docs/drafts.

[2] ProveIt draft, `ipra_certified_zero_2026-11-10.zip`, in snapshot [1]. Source of the current smallest complete checkers, initializer-based dispatch, the certified-zero normalization, the source normalizer, and attributed inherited arithmetic and logic.

[3] ProveIt draft, `ipra_bounded_initializer_2026-11-10.zip`, in snapshot [1]. Source of the tied-seed $B + \epsilon$ basis, bounded compiler, cap coherence, finite quantifier-free value graphs, internal order lemmas, and the open reflection-replacement question.

[4] ProveIt draft, `ipra_local_scopes_2026-11-10.zip`, in snapshot [1]. Tied-seed macros, finite-profile obstructions, and prior lexical-scope experiments.

[5] ProveIt draft, `ipra_arity_calculus_2026-11-10.zip`, in snapshot [1]. Unary paired formulas, triplicated closures, uniform arity-based depth, and conservative surplus-array semantics.

[6] ProveIt draft, `ipra_successor_seed_completion_2026-11-10.zip`, in snapshot [1]. The finite-range guarded-case construction adapted in the native bootstrap, with the Boolean-first selector supplied here by different arithmetic terms.

[7] ProveIt draft, `ipra_affine_supply_2026-11-10.zip`, in snapshot [1], `evidence/theory_review.md`, section 5. Contains an uncompleted B+monus suggestion and cautions about circular equality-reflection arguments.

[8] S. Salehi, "Decidable formulas of intuitionistic primitive recursive arithmetic," *Reports on Mathematical Logic* 36 (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.